Software Architecture and Design Specification

Project: File Compression Tool (Basic ZIP Implementation)

Version: 1.0

Authors: Chandrakanth V G, Debaditya Chakrabarti, Divisha Gupta, Greeshma N

Date: 02-10-2026

Status: Draft – Part 1

Revision History

Version | Date | Author | Change Summary

Version	Date	Author	Change Summary
1.0	02-10-2026	Team	Completed architecture, component design, security architecture and UML design for Part 1.
1.0	02-10-2026	Team	Reviewed API, error handling, traceability and next steps.

Approvals

Role | Name | Signature/Date

Role	Name	Signature/Date
Team Lead / Architecture	Greeshma N	Pending review
Algorithm Lead	Divisha Gupta	Pending review
Archive & I/O Lead	Debaditya Chakrabarti	Pending review
Testing & Security Lead	Chandrakanth V G	Pending review

1. Introduction

1.1 Purpose

This Software Architecture and Design Specification defines the architecture and design for the Huffman-based File Compression Tool. It translates the SRS requirements into an implementable modular design.

1.2 Scope
The design covers compression, decompression, Huffman tree/code generation, bit-level I/O, project-defined archive creation/parsing, metadata inspection, integrity validation, safe output handling, error reporting, and command-line integration.

1.3 Audience
The document is intended for the four-member development team, project guide/instructor, testers, reviewers, and future maintainers. It is used together with the SRS and Software Test Plan.

1.4 Definitions
Huffman tree, Huffman code, bit stream, archive, metadata, checksum, compression ratio, CLI, binary I/O, RTM.

2. Document Overview

2.1 How to use this document
Use Sections 3 and 4 to understand the architecture, component responsibilities, chosen pattern, security architecture, UML sequence flows, API contracts, and error-handling approach. Traceability connects design elements to SRS requirements.

2.2 Related Documents
Software Requirements Specification (SRS) v1.0; Software Test Plan (STP) v1.0; Group 14 project statement; Requirement Traceability Matrix (RTM).

3. Architecture

3.1 Goals & Constraints

Goals: lossless restoration, modularity, portability, predictable error handling, measurable compression, and secure archive validation. Constraints: Huffman coding is mandatory; implementation is C/C++; archive format is project-defined; no third-party compression engine; local file system only; no network dependency.

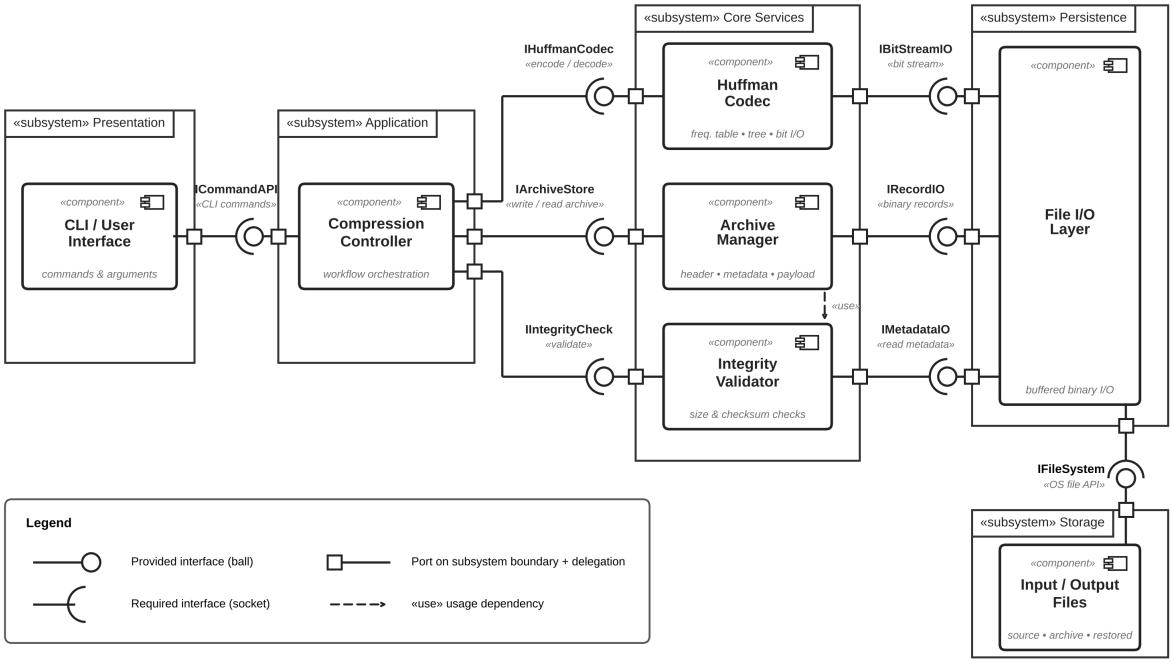
3.2 Stakeholders & Concerns

Users: simple commands and reliable output.
Project Guide/Reviewers: traceable requirements, UML/design completeness, and measurable quality.
Developers: modularity, testability, portability.
Testers: deterministic behavior, clear errors, and testable interfaces.
Maintainers: documented archive format and low coupling.

Architecture deliverables checklist: the component diagram, component descriptions, architecture pattern and rationale, technology/data-store decisions, risks/mitigations, requirement traceability, and security architecture are all included in Sections 3.3–3.9.

3.3 Component (UML) Diagram

Figure 1. UML Component Diagram



3.4 Component Descriptions

- CLI / User Interface: parses commands, validates basic arguments, and presents results.
- Compression Controller: orchestrates compression/decompression workflows and coordinates modules.
- Huffman Codec: builds frequency tables and Huffman trees, generates codes, encodes bytes, and decodes bit streams.
- Archive Manager: writes/reads archive headers, metadata, tree representation, and payload.
- File I/O Layer: performs buffered binary reads/writes and file existence/permission checks.
- Integrity Validator: checks archive magic/version, declared sizes, checksum/integrity information, and decoded output limits.
- Input/Output Files: persistent source, archive, and restored files.

3.5 Chosen Architecture Pattern and Rationale

Layered architecture with modular components is selected because the project is a small standalone utility with clear processing stages. The CLI/presentation layer, controller layer, algorithm layer, archive layer, and file I/O layer can be tested independently. A microservices architecture is rejected because it would add deployment and network complexity without benefit for a local compression utility.

3.6 Technology Stack & Data Stores

Implementation: C++17 (or later standard-compliant C++ compiler). Data structures: arrays/vectors, priority queue, tree nodes, bit buffers, and metadata structures. Storage: local binary files; no database. Development/testing tools may include GCC/Clang/MSVC, CMake/Make, PlantUML or Graphviz, and a unit-test framework where permitted.

3.7 Risks & Mitigations

Risk: malformed archive causes unsafe or incorrect restoration → Mitigation: validate magic/version/sizes/tree/checksum before trusted output.
Risk: bit-boundary errors → Mitigation: dedicated bit reader/writer and round-trip tests.
Risk: large file memory use → Mitigation: buffered streaming I/O and bounded metadata.
Risk: output overwrite → Mitigation: explicit overwrite option and pre-write existence checks.
Risk: inconsistent archive versions → Mitigation: version field and strict parser.

3.8 Traceability to Requirements

FR-CMP-01 → CLI + File I/O; FR-CMP-02/03/04/07 → Huffman Codec; FR-CMP-05 → Archive Manager; FR-CMP-06 → Compression Controller; NFR-PERF-01 → File I/O + Huffman Codec; SEC-REQ-01 → Integrity Validator; SEC-REQ-02 → Archive Manager; SAF-01/02 → File I/O + Controller.

Detailed traceability matrix (SRS requirement → architecture component → design element → verification test case):

SRS Requirement	Architecture Component	Design Element	Verification
FR-CMP-01	CLI, File I/O	Command parser and file-open checks	TC-01
FR-CMP-02	Huffman Codec	Frequency table	TC-02
FR-CMP-03	Huffman Codec	Priority queue + tree builder	TC-03
FR-CMP-04	Huffman Codec	BitWriter + encoder	TC-04
FR-CMP-05	Archive Manager	Header/tree/payload serializer	TC-05
FR-CMP-06	Controller	Statistics/reporting	TC-06
FR-CMP-07	Codec + Archive	Decoder + round-trip validation	TC-07
SEC-REQ-01	Integrity Validator	Header/checksum/size validation	TC-08

SRS Requirement	Architecture Component	Design Element	Verification
SEC-REQ-02	Archive Manager	Output path sanitization	TC-09
NFR-PERF-01	File I/O + Codec	Streaming/buffering benchmark	TC-10
FR-DEC-01	Archive Manager + Integrity Validator	Format/magic/version validation	TC-08
FR-DEC-02	Huffman Codec + Archive Manager	Decode to declared original size	TC-07
FR-DEC-03	Integrity Validator	Checksum/integrity verification	TC-08
FR-DEC-04	Archive Manager	Output path validation	TC-09
FR-INFO-01	Archive Manager	Metadata parser / info command	TC-05
FR-ERR-01	Controller + File I/O	Error/result handling	TC-08
FR-OUT-01	Archive Manager + File I/O	Validated output creation	TC-09
NFR-REL-01	Controller + Validator	Fail-closed error flow	TC-08
NFR-USE-01	CLI Controller	Help and actionable errors	TC-01
NFR-MAIN-01	All Components	Single-responsibility module interfaces	Review-01
NFR-PORT-01	All Components	Standard C++17 interfaces	Build-01
NFR-TEST-01	All Components	Unit-testable APIs	Unit-01
NFR-SAFE-01	File I/O + Archive Manager	No overwrite / cleanup	TC-09
NFR-SEC-01	Integrity Validator	Archive integrity validation	TC-08
NFR-SEC-02	Archive Manager	Path traversal protection	TC-09
NFR-SEC-03	Controller / Logger	No full-content diagnostics	TC-08
NFR-SEC-04	Validator + Decoder	Declared-size resource guard	TC-08

Verification coverage: TC-01–TC-07 cover normal functional behavior, TC-08 covers malformed/tampered archives and integrity/security validation, TC-09 covers path traversal and overwrite protection, and TC-10 covers the measurable performance requirement. Review/build checks cover maintainability, portability, and testability.

3.9 Security Architecture

Security objectives: (1) detect malformed/tampered archives before trusted output; (2) prevent unsafe path handling and accidental overwrites. Controls: strict magic/version validation, declared-size limits, checksum verification, sanitized output paths, no path traversal, no sensitive content logging, and fail-closed behavior on validation errors. STRIDE mapping: Tampering → checksum and metadata validation; Information Disclosure → limited diagnostics; Denial of Service → output-size/structure bounds; Path Abuse → output path sanitization.

Security requirements covered: SEC-REQ-01 archive structure/integrity validation; SEC-REQ-02 path safety; SEC-REQ-03 information-exposure control; SEC-REQ-04 declared-size/resource guard. Security validation is mapped to TC-08 and TC-09 in the Test Plan.

4. Design

4.1 Design Overview

The design follows a linear but testable pipeline: CLI → Compression Controller → Huffman Codec / Archive Manager → File I/O, with Integrity Validator used during archive creation and decompression. The controller owns the workflow; algorithm and persistence modules do not depend on the CLI.

4.2 UML Sequence Diagrams

Figure 2. UML Sequence Diagram – Compression Flow

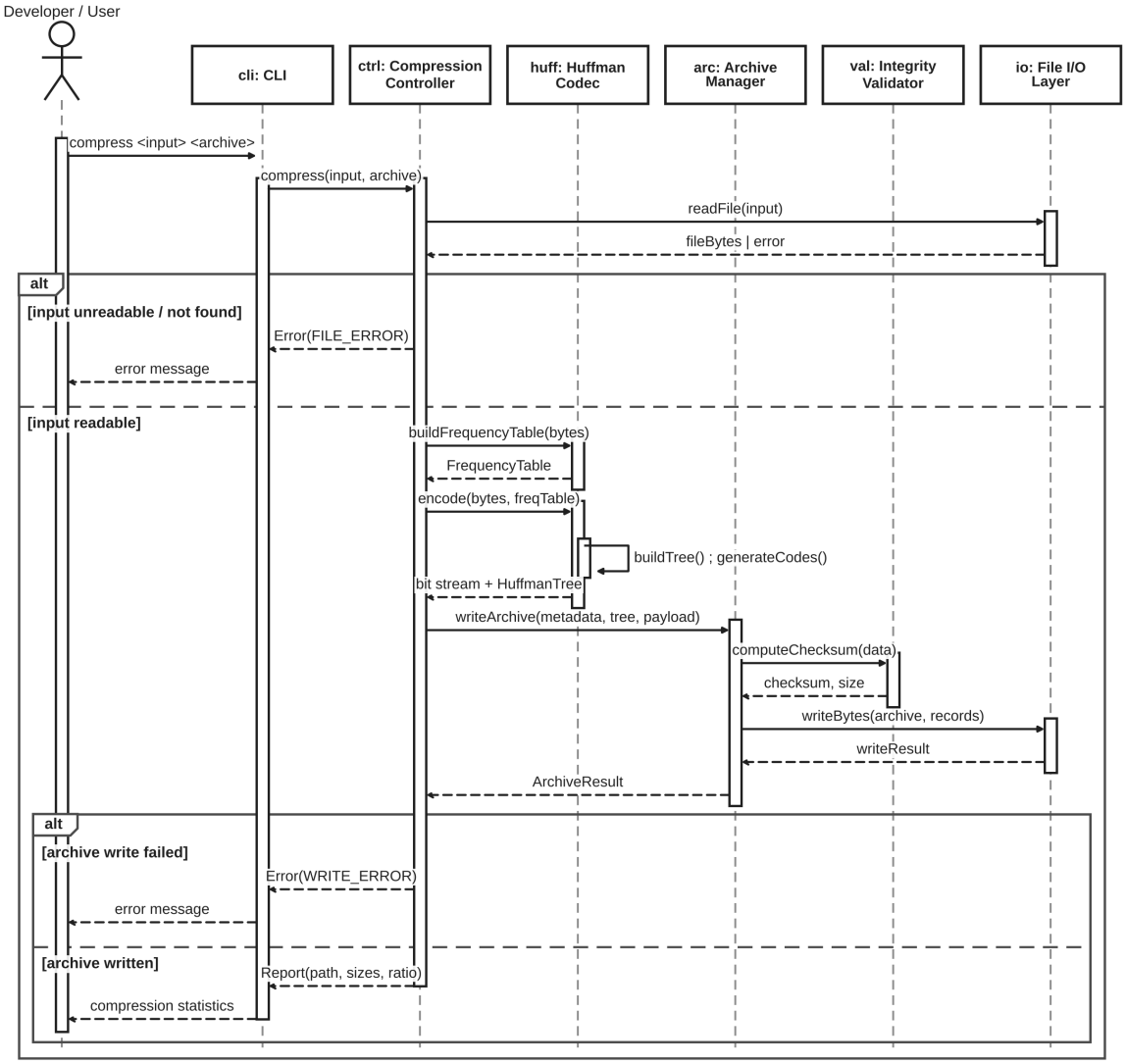


Figure 2 shows the compression flow from command input through frequency analysis, Huffman encoding, archive creation, integrity information, and final reporting. The Compression Controller owns the workflow: it reads the input through the File I/O Layer, asks the Huffman Codec to build the frequency table, the Huffman tree and the codes, and passes the encoded payload to the Archive Manager, which obtains a checksum and size from the Integrity Validator before writing the archive through File I/O. If the input cannot be read or the archive cannot be written, a typed error is returned to the CLI, which shows a clear message and a non-zero exit code.

Figure 3. UML Sequence Diagram – Decompression Flow

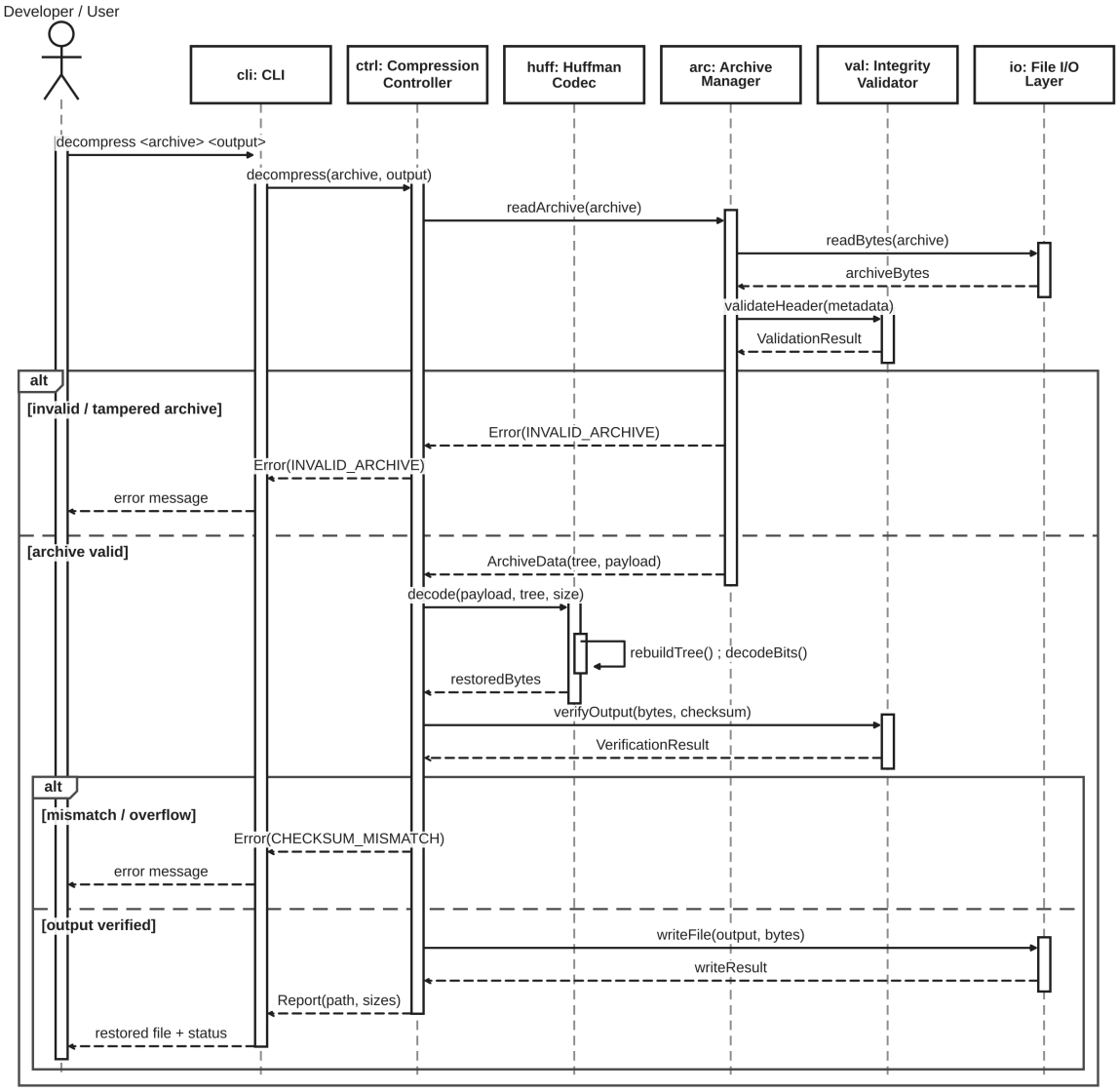


Figure 3 shows decompression, archive validation, tree reconstruction, decoding, output writing, and error reporting. Figure 4 shows the info command, which reads and validates only the archive header and reports its metadata without decoding the payload (FR-INFO-01, TC-05). Figure 5 shows the security flow for output paths: the output path is sanitized to

reject path traversal, and an existing file is never overwritten unless the overwrite option is given (SEC-REQ-02, NFR-SEC-02, NFR-SAFE-01, TC-09). Error branches in all figures are fail-closed: no output is produced after a validation failure.

Figure 4. UML Sequence Diagram – Archive Info Flow

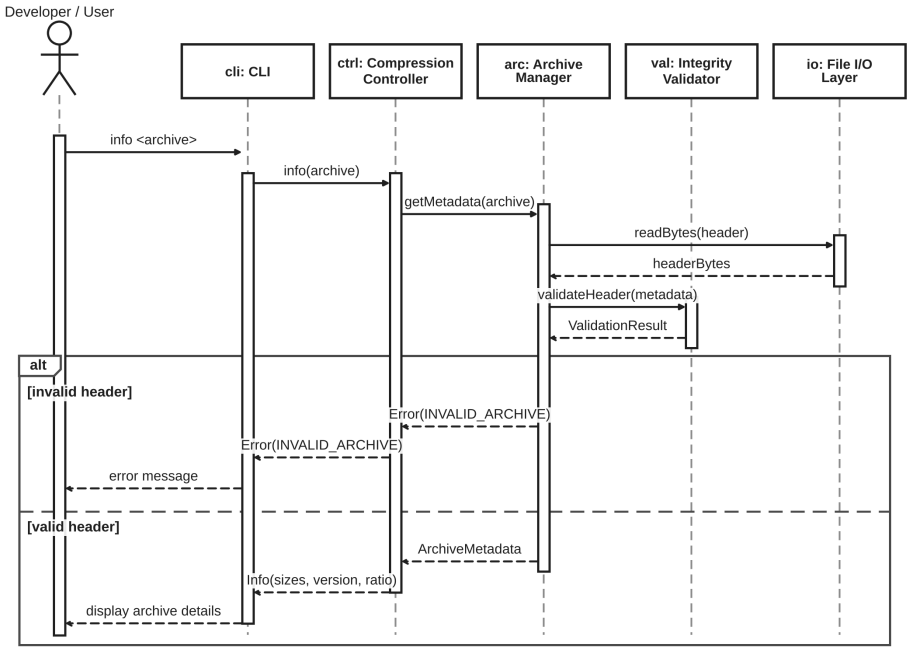
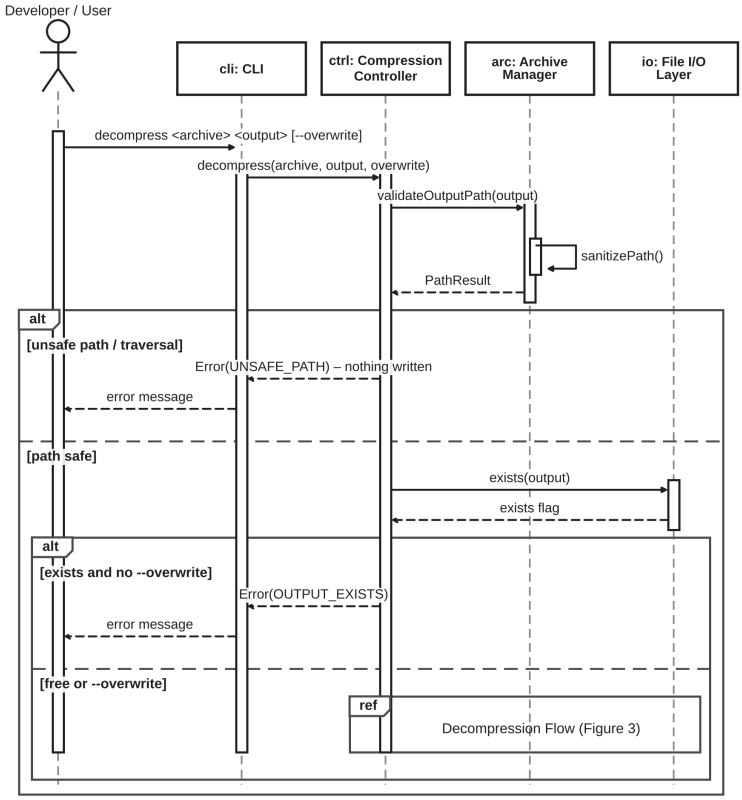


Figure 5. UML Sequence Diagram – Output Path Validation and Overwrite Protection



4.3 API Design

The internal API is designed as module-level interfaces so components can be unit tested independently. At least two component interfaces are defined below.

1) Huffman Codec API
- buildFrequencyTable(inputStream) → FrequencyTable
- buildTree(FrequencyTable) → HuffmanTree
- generateCodes(HuffmanTree) → CodeTable
- encode(inputStream, CodeTable, BitWriter) → EncodedStats
- decode(BitReader, HuffmanTree, expectedSize, outputStream) → DecodeStats

2) Archive Manager API
- writeArchive(outputPath, metadata, tree, payload) → ArchiveResult
- readArchive(inputPath) → ArchiveData / error
- validateHeader(metadata) → ValidationResult
- getMetadata(inputPath) → ArchiveMetadata

CLI commands: `compress <input> <archive>`, `decompress <archive> <output>`, `info <archive>`.

4.4 Error Handling, Logging & Monitoring

Errors use typed/result-style return values or exceptions only at module boundaries, with a single user-facing error formatter in the controller. Categories include invalid arguments, file-not-found, permission denied, invalid archive magic/version, malformed Huffman metadata, checksum mismatch, decoded-size overflow, and output-write failure. Diagnostic logs shall never contain complete file contents. Useful metrics include input/output sizes, compression ratio, operation duration, and validation result.

4.5 UX Design

The command-line interface shall use consistent command syntax, concise status messages, explicit error causes, and non-zero process exit codes on failure. Help output shall list commands and required arguments. Success output shall report the output path and compression statistics. The design avoids requiring knowledge of Huffman internals for normal users.

4.6 Open Issues & Next Steps

Implement and freeze the archive binary format; complete module implementation; add unit/integration/system tests; benchmark reference files; review security validation; and update the RTM with final code/test identifiers. Possible future enhancements include standard ZIP/DEFLATE interoperability, GUI support, multi-file archives, and optional password-based encryption.

5. Appendices

5.1 Glossary: Huffman tree – binary prefix-code tree; Bit stream – encoded sequence of bits; Archive – project-defined compressed file; Checksum – integrity value; CLI – command-line interface; RTM – requirement traceability matrix.

5.2 References: IEEE 29148-2018; IEEE 1016-2009; Huffman (1952); project SRS; project STP.

5.3 Tools: Graphviz/PlantUML for UML diagrams, C++17 toolchain, Git, CMake/Make, and a unit-test framework where permitted.